



Edge AI na prática com Arduino UNO Q

Do Blink ao agente de borda

Slides dos laboratórios 0 a 15

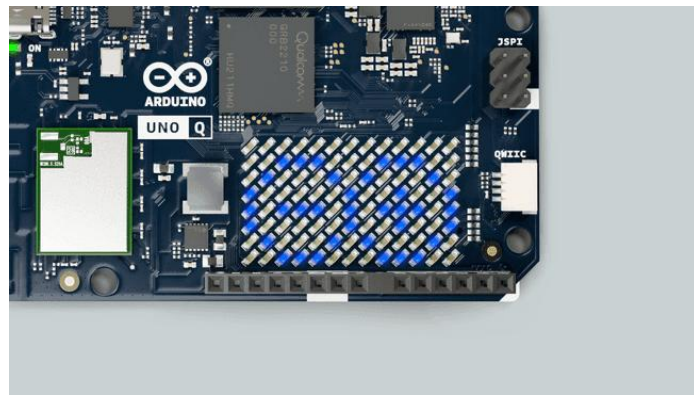


Abertura

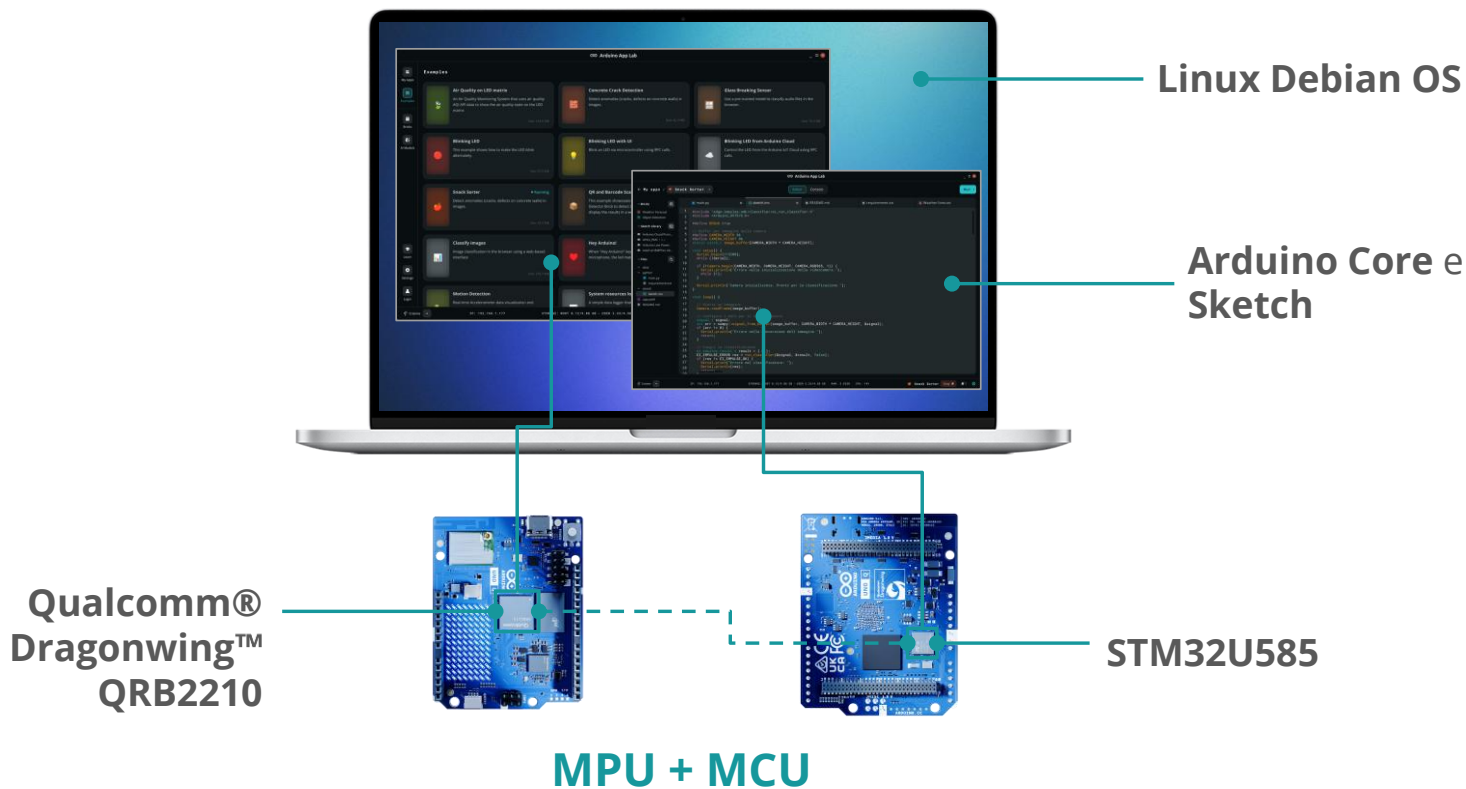
Edge AI e o Arduino UNO Q

O que é Edge AI

- **Inferência no dispositivo** que coletou o dado, sem ida e volta à nuvem
- **Latência:** um drone ou uma máquina não podem esperar a rede
- **Privacidade:** o vídeo e os dados de saúde não saem do aparelho
- **Disponibilidade:** funciona com a rede fora do ar
 - Hoje você roda modelos de IA em uma placa do tamanho da palma da mão



Arquitetura de duplo cérebro: **Linux encontra Arduino**



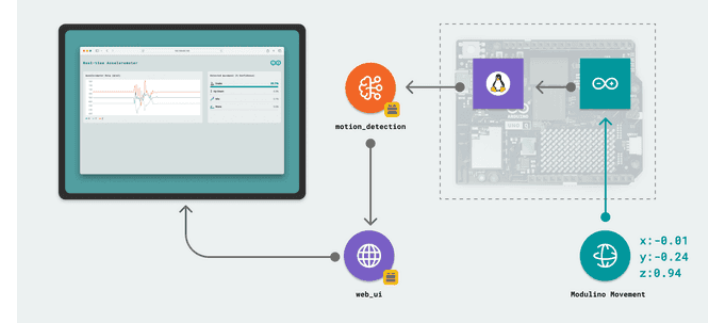
Dois processadores, cada um no seu papel

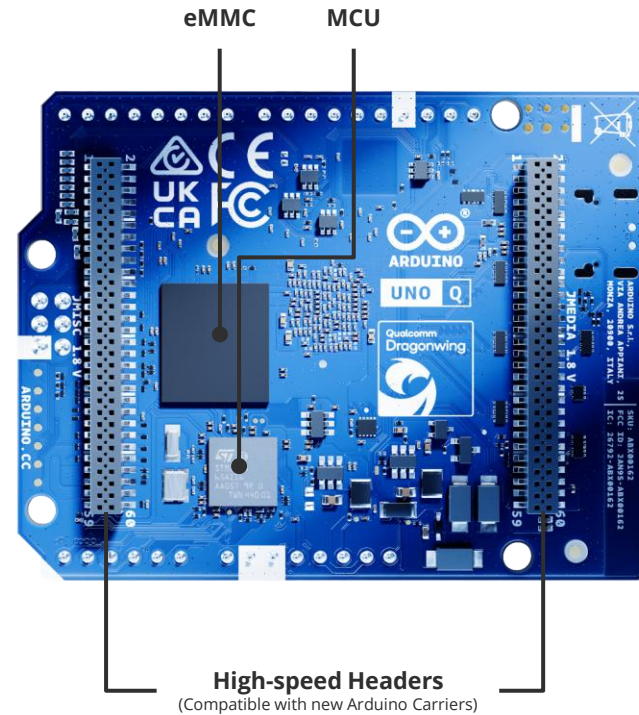
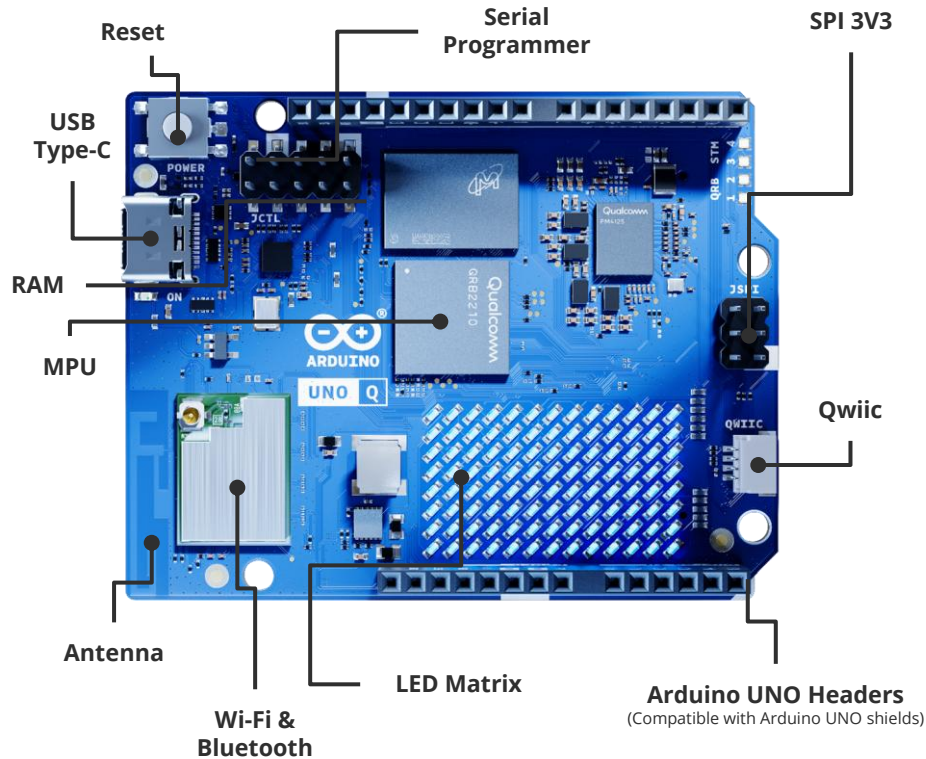
	MPU Qualcomm QRB2210	MCU STM32U585
Sistema	Linux Debian	Zephyr, tempo real
Você programa em	Python (python/main.py)	C++ (sketch/sketch.ino)
Bom para	IA, rede, interface web	pinos, sensores, temporização
Recursos	4 núcleos a 2 GHz, 4 GB de RAM	160 MHz, 768 KB de flash para o sketch
Ligação entre os dois	Bridge: o Python chama funções do sketch pelo nome	



Sentir, pensar, agir

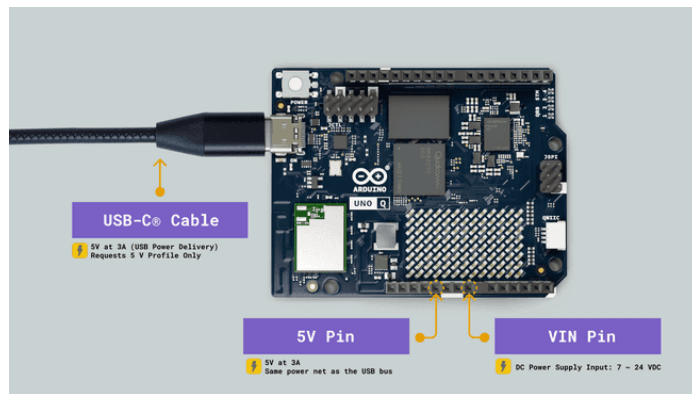
- **Sentir:** capturar o mundo com um sensor, uma câmera, uma foto
- **Pensar:** um modelo de IA interpreta o dado
- **Agir:** acender um LED, mover um motor, avisar alguém
 - Todo sistema embarcado inteligente repete esse ciclo. Hoje você monta o ciclo inteiro.





Uma placa, uma porta USB-C

- A mesma porta serve para **alimentar**, para falar com o **notebook** e para ligar **periféricos**
- Mas não tudo ao mesmo tempo
- **Sem câmera:** cabo USB-C direto no notebook
- **Com webcam:** hub USB-C com Power Delivery; a fonte e a webcam vão no hub; o notebook fala com a placa pelo **Wi-Fi**
- **Com celular como câmera:** o vídeo vai pelo Wi-Fi e a placa pode ficar no cabo
 - A placa pode reiniciar na troca do cabo. Ela leva cerca de 1 minuto para voltar.



Um app do App Lab tem duas metades

```
meu-app/  
  app.yaml          # nome, icone e lista de Bricks  
  python/main.py    # aplicacao Python (roda na MPU)  
  sketch/sketch.ino # codigo C++ do MCU  
  sketch/sketch.yaml # plataforma e bibliotecas do sketch (gerado pelo App Lab)  
  assets/           # so em apps com interface web: HTML, CSS, JS  
  README.md         # so em alguns apps
```

O App Lab compila o sketch, grava no MCU e sobe o Python. Cada **Run** leva de 10 segundos a 2 minutos; os mais longos são os que compilam o sketch.



O material do workshop

- **Manual:** o passo a passo completo dos 16 laboratórios, em PDF
- **Estes slides:** o resumo de cada laboratório, com o código principal e os números de referência
- **Kit de apps:** um app pronto por laboratório, instalado na placa; em **My Apps** eles começam por "WS", como **WS Lab 1 Blink**
- **Onde está tudo:** github.com/joaodullius/workshop-edge-ai-uno-q
- **Para copiar código:** use a pasta **codigo/** do repositório, não o PDF, que quebra as linhas longas
 - Em uma aula curta fazemos só alguns laboratórios, ou só algumas partes. O resto você faz depois, com uma placa e este material.



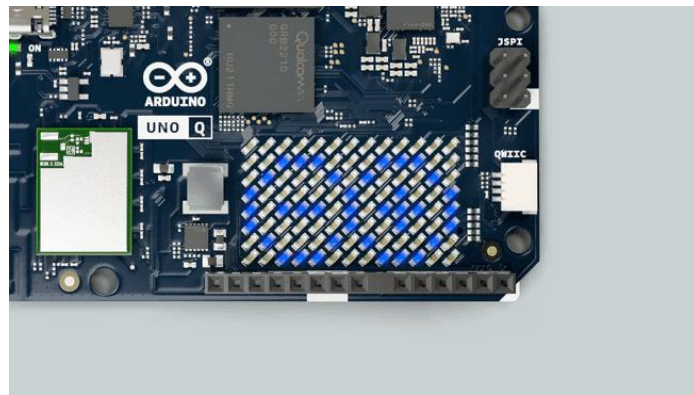


Laboratório 0

Preparando-se

Neste laboratório

- Criar as contas: **Arduino** e **Edge Impulse**
- Instalar o **Arduino App Lab** no computador
- Ligar a placa, dar um nome a ela e configurar o **Wi-Fi**
- Testar os dois acessos por terminal: **ADB** pelo cabo e **SSH** pela rede
- Instalar o conector do **Arduino Cloud**
 - Em um workshop curto as placas chegam prontas: o instrutor faz este laboratório antes



Os dois terminais da placa

terminal do computador; depois de adb shell ou ssh, os comandos rodam na placa

```
adb devices
adb shell

ssh arduino@<nome-da-placa>.local

uname -a          # Linux ... aarch64
cat /etc/os-release # Debian GNU/Linux 13 (trixie)
free -h           # cerca de 3,6 GiB de RAM
# armazenamento interno (eMMC): / com ~10 GB e /home/arduino com ~18 GB
df -h
exit              # volta ao computador
```

ADB exige a placa no cabo USB. **SSH** exige a placa e o computador na mesma rede.



Laboratório 1

Seu primeiro app

Neste laboratório

- **Parte A:** rodar o Blink, com o Python mandando e o microcontrolador obedecendo
- **Parte B:** entender os arquivos de um app
- **Parte C:** fazer o mesmo pela linha de comando
- **Parte D:** controlar o LED pelo Arduino Cloud



Abra o Blink

- 1 No App Lab, abra **Examples**, escolha **Blink LED from Python** e clique em **Copy and edit app**. Com o kit do workshop: abra **WS Lab 1 Blink** em **My Apps**.

VOCÊ DEVE VER

À esquerda, o painel **Files** com `python/main.py`, `sketch/sketch.ino` e `app.yaml`.

SE DER ERRADO

A placa não aparece no App Lab? Confira o cabo (tem que ser de dados) e espere 1 minuto depois de ligar.



Leia o Python

2

Abra **python/main.py**. A função **loop** roda repetidamente e chama **set_led_state**, que não está no Python: está no sketch.

VOCÊ DEVE VER

Bridge.call leva o valor **True** ou **False** até o microcontrolador.

python/main.py

```
from arduino.app_utils import *
import time

led_state = False

# Esta funcao e chamada repetidamente pelo App.run()
def loop():
    global led_state
    time.sleep(1)
    led_state = not led_state
    # Chama a funcao "set_led_state" definida no sketch,
    # passando True ou False
    Bridge.call("set_led_state", led_state)

App.run(user_loop=loop)
```



Leia o sketch

sketch/sketch.ino

```
#include "Arduino_RouterBridge.h"

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    Bridge.begin();
    Bridge.provide("set_led_state", set_led_state); // obrigatorio para usar a Bridge // expoe a funcao para o Python
}

void loop() {
}

void set_led_state(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH); // LED_BUILTIN acende em LOW
}
```

Bridge.provide dá um nome à função. É por esse nome que o Python a chama.



Rode

- 3 Clique em **Run**, no canto superior direito. Abra o console, abaixo do editor, e acompanhe: compilação, gravação do sketch, início do Python.

VOCÊ DEVE VER

O LED vermelho da placa mudando de estado a cada segundo: um segundo aceso, um apagado.

SE DER ERRADO

Demorou mais de 3 minutos? Clique em Stop e rode de novo.



Mude e rode de novo

4

Troque `time.sleep(1)` por `time.sleep(0.2)` no Python e clique em **Run** de novo.

VOCÊ DEVE VER

O LED piscando cinco vezes mais rápido. Você mudou o comportamento do hardware sem tocar no sketch.



O que você acabou de fazer

- O **Python** decidiu quando acender; o **microcontrolador** acendeu
- A **Bridge** levou a ordem de um processador ao outro
- No UNO Q, **LED_BUILTIN** acende com **LOW**
- O laço da aplicação vai em **App.run(user_loop=...)**



O mesmo, pela linha de comando

terminal da placa

```
# lista apps e exemplos
arduino-app-cli app list
arduino-app-cli app start examples:core-and-foundational/01-led-blink/03-blinking-an-led-from-python
# traz tambem a saida de execucoes anteriores
arduino-app-cli app logs examples:core-and-foundational/01-led-blink/03-blinking-an-led-from-python --all
arduino-app-cli app stop examples:core-and-foundational/01-led-blink/03-blinking-an-led-from-python
```

O **arduino-app-cli** é a ferramenta que o App Lab usa por baixo. Tudo o que a interface faz, ele faz.



O LED pelo Arduino Cloud

D

Conecte a placa à Cloud em **Settings**, crie uma Thing com a variável **led** e rode o exemplo **Blinking LED from Arduino Cloud**.

VOCÊ DEVE VER

O interruptor do painel na Cloud acende e apaga o LED da placa.

SE DER ERRADO

Nenhuma credencial vai no código: a conexão é da placa, feita em Settings.

python/main.py

```
from arduino.app_bricks.arduino_cloud import ArduinoCloud
from arduino.app_utils import App, Bridge

# A conexao com o Cloud e feita pelo servico da placa;
# nenhuma credencial aqui
iot_cloud = ArduinoCloud()

def led_callback(client: object, value: bool):
    # Chamado sempre que a variavel "led" muda na nuvem
    print(f"LED blink value updated from cloud: {value}")
    # repassa ao sketch, que controla o pino
    Bridge.call("set_led_state", value)

iot_cloud.register("led", value=False, on_write=led_callback)

App.run()
```



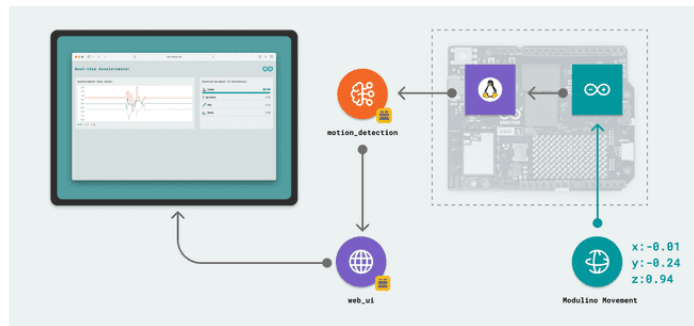


Laboratório 2

Bricks e IA em uma foto

Neste laboratório

- **Parte A:** uma página web servida pela placa, com o Brick **Web UI**
- **Parte B:** onde ler a documentação de cada Brick
- **Parte C:** o primeiro modelo de IA, rodando sobre uma **foto** enviada pelo navegador
- **Parte D:** por que cada Brick roda em um container
 - Não precisa de câmera. Tenha uma ou duas fotos JPG ou PNG de até 500 KB.



O que é um Brick

- Um bloco pronto que você adiciona ao app: interface web, modelo de IA, nuvem, banco de dados
- Cada Brick aparece no **app.yaml** e vira um **import** no Python
- Os Bricks de IA rodam em um container separado, com o modelo dentro
 - Na primeira vez que um Brick é usado, a placa baixa o container. Pode levar alguns minutos.



Uma página servida pela placa

A

Copie o exemplo **Simple web page** com o nome **brick-explorer**. O **main.py** cria o Brick; a página fica em **assets/index.html**.

python/main.py

```
from arduino.app_utils import App
from arduino.app_bricks.web_ui import WebUI

# Serve os arquivos da pasta assets/ em
# http://<nome-da-placa>.local:7000
web_ui = WebUI()

App.run()
```

VOCÊ DEVE VER

Em **http://<nome-da-placa>.local:7000**, a sua página, vinda da placa.

SE DER ERRADO

Trocou o **index.html** com o app rodando e a página não mudou? Pare o app e rode de novo.



Rode o app da foto

C Em **Inspirations**, copie o exemplo **Detect objects on images** com o nome **detect-foto** e clique em **Run**. Com o kit: abra **WS Lab 2 IA em uma foto**.

VOCÊ DEVE VER

No console, o endereço da página: **`http://<ip-da-placa>:7000`**.

SE DER ERRADO

Primeira vez neste app? A placa pode estar baixando o container do modelo. Espere.



Envie uma foto

- C** Abra **`http://<nome-da-placa>.local:7000`**. Escolha uma foto com pessoas ou objetos comuns e clique em **Run Detection**. Depois mexa no controle **Confidence** e clique em **Run Again**.

VOCÊ DEVE VER

A foto de volta, com uma caixa em volta de cada objeto, o nome da classe e a confiança.

SE DER ERRADO

O nome **`.local`** não abre? Use o endereço IP da etiqueta da placa. A página só aceita JPG ou PNG de até 500 KB.



O resultado na página

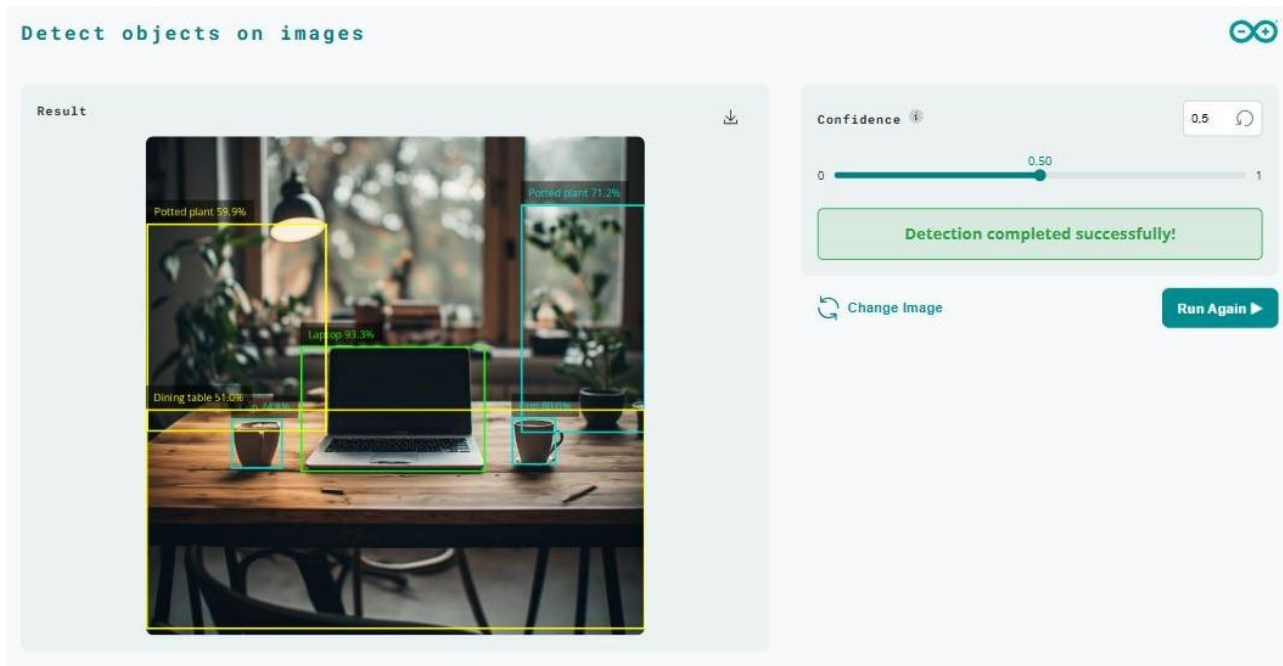


Foto de referência: notebook com 93% de confiança, duas plantas em vaso, duas xícaras e a mesa.



O coração do app: duas chamadas

python/main.py

```
from arduino.app_bricks.object_detection import ObjectDetection

object_detection = ObjectDetection()

# dentro da funcao que recebe a foto do navegador:
results = object_detection.detect(pil_image, confidence=confidence)
img_with_boxes = object_detection.draw_bounding_boxes(pil_image, results)

# results = {"detection": [{"class_name": "person", "confidence": 87.3,
#                           "bounding_box_xyxy": [x1, y1, x2, y2]}, ...]}
```

Sentir foi a foto. Pensar foi **detect**. Falta agir.



Quanto tempo a placa leva para pensar?



No **main.py**, logo abaixo da linha **diff = ...**, vão estas duas linhas. **No app do kit elas já estão lá:** só localize e leia o console. Sem o kit: cole com o mesmo recuo, Stop, Run, recarregue a página e envie a foto de novo.

python/main.py

```
n_obj = len(results.get("detection", [])) if results else 0
print(f"Deteccao: {n_obj} objetos em {diff:.0f} ms")

# no console do App Lab:
# Deteccao: 6 objetos em 1149 ms
```

VOCÊ DEVE VER

No console, uma linha como **Deteccao: 6 objetos em 1149 ms**. Pouco mais de **1 segundo** por foto.

SE DER ERRADO

A linha não aparece? Confira o recuo (oito espaços), se você parou e rodou o app de novo, e recarregue a página.



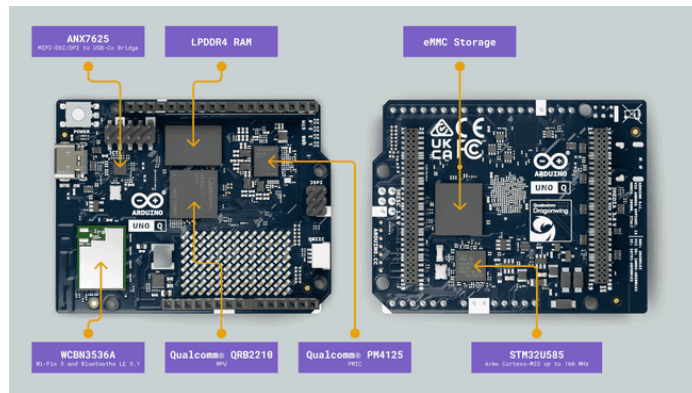


Laboratório 3

Os dois mundos: Linux e o MCU

Neste laboratório

- **Parte A:** explorar o lado Linux pelo terminal
- **Parte B:** editar o sketch no App Lab: um botão acende o LED
- **Parte C:** entender a Bridge: o sketch expõe, o Python chama
- **Parte D:** o Python pisca o LED do MCU



Um computador Linux de verdade

terminal da placa

```
uname -a                # kernel 6.16, arquitetura aarch64
cat /etc/os-release      # Debian 13 (trixie)
nproc                    # 4 núcleos
free -h                  # 3,6 GiB de RAM e 1,8 GiB de swap
ls /home/arduino/ArduinoApps/ # seus apps
ls /var/lib/arduino-app-cli/examples/ # exemplos do App Lab
ps aux | grep -E "python|docker" | head # processos dos apps e do Docker
```

Debian, quatro núcleos, 3,6 GiB de RAM. O seu Python roda aqui, dentro de um container.



O sketch expõe uma função

sketch/sketch.ino

```
// sketch/sketch.ino - expõe set_led ao Python
#include <Arduino_RouterBridge.h>

void set_led(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);
}

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    Bridge.begin();
    Bridge.provide("set_led", set_led);    // agora o Python pode chamar "set_led"
}

void loop() {
    // pode ficar vazio: a Bridge atende as chamadas em segundo plano
}
```

Sem a linha **Bridge.provide**, o Python não enxerga a função.



O Python pisca o LED do MCU

D

Crie um app vazio chamado **mcu-basics** pelo botão +. Cole o sketch do slide anterior em **sketch/sketch.ino** e este código em **python/main.py**. Clique em **Run**.

VOCÊ DEVE VER

O LED piscando, com a temporização vinda do Python.

SE DER ERRADO

Function not found? O nome em **Bridge.provide** e em **Bridge.call** tem que ser idêntico.

python/main.py

```
# python/main.py - pisca o LED do MCU a partir do Python
from arduino.app_utils import App, Bridge
import time

state = False

def loop():
    global state
    state = not state
    Bridge.call("set_led", state)
    print("LED do MCU:", "ON" if state else "OFF")
    time.sleep(1)

# o App Lab chama loop() repetidamente
App.run(user_loop=loop)
```



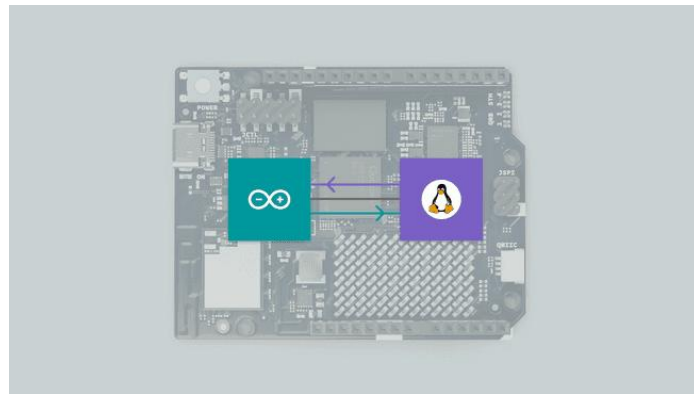


Laboratório 4

Comunicação entre processadores

Neste laboratório

- **Parte A:** o Python chama funções do sketch e recebe a resposta
- **Parte B:** medir quanto custa uma chamada
- **Parte C:** um laço que lê um sensor e aciona o LED
- **Parte D:** o caminho inverso: o MCU avisa o Python
- **Parte E:** desafio: a foto que acende o LED



A Bridge em três verbos

Verbo	Quem usa	O que faz
<code>Bridge.provide("nome", funcao)</code>	sketch e Python	expõe uma função pelo nome
<code>Bridge.call("nome", args)</code>	Python	chama a função do sketch e espera a resposta
<code>Bridge.notify("nome", args)</code>	sketch e Python	chama sem esperar resposta



O sketch expõe três funções

sketch/sketch.ino

```
float read_sensor() {
    return analogRead(A0) * 3.3 / 1024.0;    // tensão em A0, em volts
}

void set_led(bool state) {
    digitalWrite(LED_BUILTIN, state ? LOW : HIGH);    // LED_BUILTIN é ativo em LOW
}

int add_numbers(int a, int b) {
    return a + b;
}

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(A0, INPUT);
    Bridge.begin();                                // inicializa a Bridge
    Bridge.provide("read_sensor", read_sensor);      // expoe cada funcao pelo nome
    Bridge.provide("set_led", set_led);
    Bridge.provide("add_numbers", add_numbers);
}

void loop() {}    // vazio: a Bridge atende as chamadas em segundo plano
```

Crie o app vazio **bridge-test** pelo botão +. Com o kit: abra **WS Lab 4 Bridge**.



O Python chama o microcontrolador

A

Leia o **main.py**: ele soma no MCU, lê o pino A0 e acende o LED por 1 segundo. Clique em **Run**.

VOCÊ DEVE VER

7 + 3 = 10, uma tensão entre 0 e 3,3 V, e o LED aceso por 1 segundo.

SE DER ERRADO

Sem nada ligado em A0, a leitura é ruído. Está certo.

python/main.py

```
# python/main.py - chama funcoes do sketch
from arduino.app_utils import App, Bridge
import time

# Passa parametros e recebe o retorno
print("7 + 3 =", Bridge.call("add_numbers", 7, 3))

# Le o sensor
voltage = Bridge.call("read_sensor")
print(f"Tensao no sensor: {voltage:.2f} V")

# Controla o LED
Bridge.call("set_led", True)
time.sleep(1)
Bridge.call("set_led", False)

App.run()
```



Quanto custa uma chamada?

B

Este bloco mede 100 chamadas seguidas. **No app do kit ele já está lá**, no **main.py**: leia o resultado no console. Sem o kit: cole logo acima da última linha, **App.run()**, e rode de novo.

VOCÊ DEVE VER

Média perto de **7 ms** por chamada, com pouca variação.

python/main.py

```
import statistics

times = []
print("Medindo a latencia da Bridge (100 chamadas)...")
for _ in range(100):
    t0 = time.perf_counter()
    Bridge.call("read_sensor")
    # em ms
    times.append((time.perf_counter() - t0) * 1000)

print(f"Media: {statistics.mean(times):.2f} ms")
print(f"Min: {min(times):.2f} ms")
print(f"Max: {max(times):.2f} ms")
print(f"Desvio padrao: {statistics.stdev(times):.2f} ms")
```



A Bridge em números

7 ms

uma chamada **Bridge.call**,
ida e volta

45 Hz

laço com três chamadas
por ciclo

120 Hz

laço com uma chamada
agrupada

Medido na placa. Agrupar o trabalho em uma função do MCU quase triplica a frequência do laço.



Um laço que sente e age



Substitua o **main.py** por este monitor contínuo. O laço vai dentro de uma função passada a **App.run**.

VOCÊ DEVE VER

A tensão impressa no console, e o LED aceso quando ela passa do limiar.

SE DER ERRADO

Sem potenciômetro, toque o pino A0 com o dedo: a leitura muda.

python/main.py

```
# python/main.py - monitora A0 e acende o LED acima do
# limiar
from arduino.app_utils import App, Bridge
import time

THRESHOLD = 1.5    # volts

def loop():
    voltage = Bridge.call("read_sensor")
    if voltage > THRESHOLD:
        Bridge.call("set_led", True)
        print(f"ALERTA: {voltage:.2f} V acima do limiar")
    else:
        Bridge.call("set_led", False)
        print(f"Normal: {voltage:.2f} V")
        time.sleep(0.5)

App.run(user_loop=loop)
```



O caminho inverso

D No sketch, substitua o **loop()** vazio por este. No Python, registre **on_tick** com **Bridge.provide**.

VOCÊ DEVE VER

A cada segundo, uma linha no console vinda do microcontrolador.

SE DER ERRADO

redefinition of 'void loop()'? O sketch ficou com duas funções **loop**.

sketch/sketch.ino

```
unsigned long last = 0;

void loop() {
    if (millis() - last >= 1000) {
        last = millis();
        // avisa o Python, sem esperar
        Bridge.notify("on_tick", (int)(millis() / 1000));
    }
}

# python/main.py
def on_tick(seconds: int):
    print(f"MCU ligado ha {seconds} s")

# expoe a funcao ao sketch
Bridge.provide("on_tick", on_tick)
```





Desafio

A foto que acende o LED

Feche o ciclo: sentir, pensar, agir

- E** Junte o Laboratório 2 e a Bridge. No app da foto, depois de **detect**, descubra se há uma **person** e mande o MCU acender o LED com **Bridge.call**. Com o kit: abra **WS Lab 4 Desafio (partida)**, que já tem o sketch do LED.

VOCÊ DEVE VER

Foto com pessoa: LED aceso. Foto sem pessoa: LED apagado.

SE DER ERRADO

Dica 1: **results["detection"]** é uma lista; cada item tem **class_name**, em minúsculas: **person**, embora a página mostre **Person**. Dica 2: o app precisa de um sketch que exponha **set_led_state**. Sem o kit, o próximo slide mostra como dar um sketch ao seu **detect-foto**.



O app da foto ganha um sketch

terminal da placa (adb shell ou SSH)

```
cp -r  
  /var/lib/arduino-app-cli/examples/core-and-foundational/01-led-blink/03-blinking-an-led-from-python/sketch  
  ~/ArduinoApps/detect-foto/  
ls ~/ArduinoApps/detect-foto/sketch  
  
# o ls deve mostrar: sketch.ino sketch.yaml
```

Os exemplos copiados não têm a pasta **sketch/**. Ela precisa de dois arquivos, **sketch.ino** e **sketch.yaml**: copie a do Blink. Se o seu app tem outro nome, troque **detect-foto** nos dois comandos.



Gabarito: três linhas

python/main.py, logo acima da linha `img_with_boxes = ...`

```
peessoa = any(d.get("class_name") == "person" for d in results.get("detection", []))
Bridge.call("set_led_state", peessoa)
print("Pessoa na foto:", peessoa, "-> LED", "aceso" if peessoa else "apagado")
```

```
# no console do App Lab:
# Deteccao: 1 objetos em 1106 ms
# Pessoa na foto: True -> LED aceso
# Deteccao: 6 objetos em 1149 ms
# Pessoa na foto: False -> LED apagado
```

O modelo leva mais de 1 segundo para pensar. A Bridge leva 7 ms para agir.



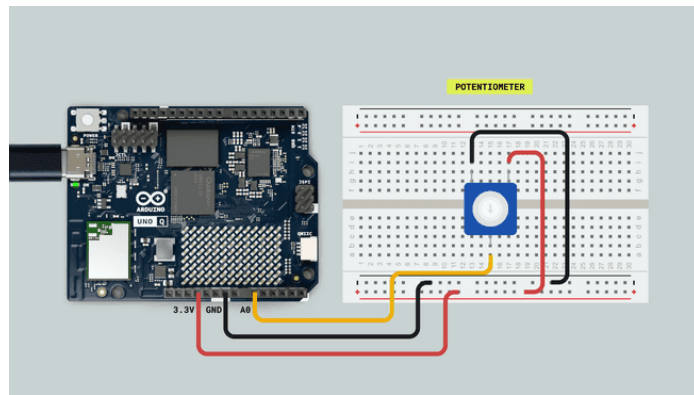


Laboratório 5

Laço de controle entre domínios

Neste laboratório

- **Parte A:** adicionar a biblioteca Servo ao sketch
- **Parte B:** um laço simples: ler, decidir, atuar
- **Parte C:** três formas de fechar o mesmo laço, e quanto cada uma rende
 - Sem servo e sem potenciômetro o laboratório roda igual: o LED serve de atuador



O ciclo inteiro no MCU, em uma chamada

sketch/sketch.ino

```
// Ciclo completo em uma unica chamada: le, decide, atua e devolve a leitura
int sense_and_act(int threshold) {
    int raw = analogRead(A0);
    // multiplica antes: divisao inteira
    myServo.write(constrain(raw * 180 / 1024, 0, 180));
    digitalWrite(LED_BUILTIN, raw > threshold ? LOW : HIGH);
    return raw;
}
```

Trecho do sketch: **myServo** e **threshold** vêm do resto do arquivo. O sketch precisa da biblioteca **Servo** (botão **Add Sketch Library**).

Meça as três formas



Com o kit: abra **WS Lab 5 Laço de controle** e clique em **Run**. Sem o kit: Lab 5, Passo 6. O **main.py** cronometra 200 ciclos de cada forma.

VOCÊ DEVE VER

Três linhas no console: A perto de 22 ms, B perto de 8 ms, C abaixo de 0,1 ms.

SE DER ERRADO

Sem servo e sem potenciômetro os números são os mesmos: o LED serve de atuador.

python/main.py

```
# A) tres chamadas por ciclo
def three_calls():
    raw = Bridge.call("read_sensor_raw")
    Bridge.call("set_servo", int(raw / 1024 * 180))
    Bridge.call("set_led", raw > 512)

# B) uma chamada agrupada, com retorno
def one_call():
    Bridge.call("sense_and_act", 512)

# C) uma chamada agrupada, sem esperar
def one_notify():
    Bridge.notify("sense_and_act", 512)

run_loop("A) 3 x call", three_calls)
run_loop("B) 1 x call", one_call)
run_loop("C) 1 x notify", one_notify)
```



Três formas de fechar o laço

Forma	Tempo por ciclo	Frequência	Observação
A) três Bridge.call	22,4 ms	45 Hz	ler, decidir e atuar em chamadas separadas
B) uma Bridge.call agrupada	8,4 ms	120 Hz	o MCU faz o ciclo inteiro
C) uma Bridge.notify	0,08 ms no Python	não mede o laço	não espera resposta do MCU



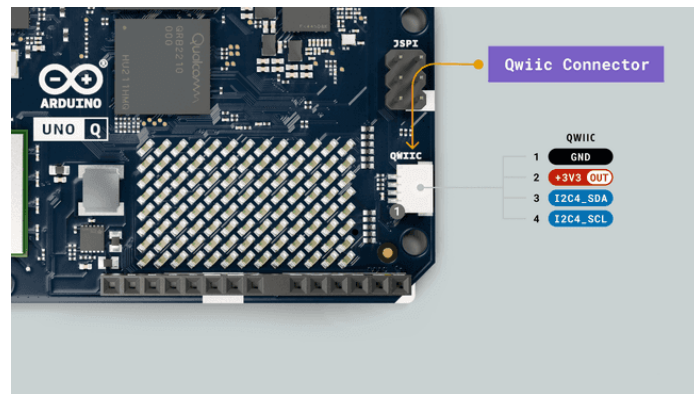


Laboratório 6

Sensores Modulino e painel na nuvem

Neste laboratório

- **Parte A:** o primeiro sensor: a biblioteca **Arduino_Modulino** e o Thermo
- **Parte B:** dois sensores encadeados no mesmo cabo
- **Parte C:** resumir localmente: média móvel e alertas
- **Partes D e E:** publicar no Arduino Cloud e montar o painel
 - Modulino só existe no sketch. O Python recebe as leituras pela Bridge.



A biblioteca e as sete dependências

sketch/sketch.yaml

```
profiles:
  default:
    platforms:
      - platform: arduino:zephyr
    libraries:
      - Arduino_Modulino (0.7.0)
      - dependency: ArduinoGraphics (1.1.4)
      - dependency: Arduino_HS300x (1.0.0)
      - dependency: Arduino_LPS22HB (1.0.2)
      - dependency: Arduino_LSM6DSOX (1.1.2)
      - dependency: Arduino_LTR381RGB (1.0.0)
      - dependency: STM32duino VL53L4CD (1.0.5)
      - dependency: STM32duino VL53L4ED (1.0.1)
  default_profile: default
```

Adicione com **Add Sketch Library** e confira o arquivo. Sem as dependências, a compilação para.



O sensor é lido no MCU

sketch/sketch.ino

```
void setup() {  
  Bridge.begin();  
  Modulino.begin(Wire1);           // barramento Qwiic do UNO Q  
  thermo.begin();  
}  
  
void loop() {  
  unsigned long now = millis();  
  if (now - previousMillis >= interval) {  
    previousMillis = now;  
    float celsius = thermo.getTemperature();  
    float humidity = thermo.getHumidity();  
    // Envia as duas leituras ao Python, sem esperar resposta  
    Bridge.notify("record_sensor_samples", celsius, humidity);  
  }  
}
```

Bridge.notify envia as leituras ao Python sem esperar resposta.



O Python resume e publica

D

As leituras chegam em um callback. As médias vão para o Arduino Cloud a cada 30 segundos; os alertas, na hora.

VOCÊ DEVE VER

No console, uma linha **Cloud: ...** a cada 10 leituras. No painel da Cloud, a temperatura média atualizando a cada 30 segundos.

SE DER ERRADO

A placa precisa estar conectada à Cloud: App Lab, **Settings, Arduino Cloud, Connect device.**

python/main.py

```
# a conexao e feita pelo servico da placa; sem credenciais
# aqui
cloud = ArduinoCloud()

# interval=30: o valor atual e publicado a cada 30 s
cloud.register("avg_temperature", value=0.0, interval=30)
cloud.register("avg_distance", value=0.0, interval=30)
# sem interval: publicado quando muda
cloud.register("alert_count", value=0)
cloud.register("status", value="OK")
```



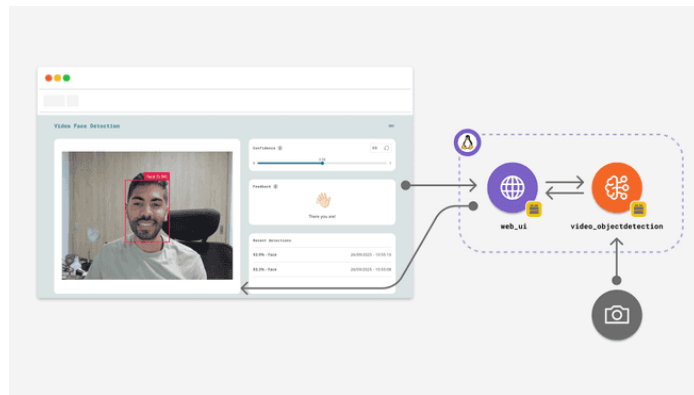


Laboratório 7

Câmera e captura de imagem

Neste laboratório

- **Parte A:** montar a placa com hub USB-C e webcam; o acesso passa a ser pela rede
- **Parte B:** conferir se o Linux enxergou a câmera
- **Parte C:** capturar um quadro e medir a taxa de captura
- **Parte D:** o que há dentro de uma imagem: pixels, cor e memória
 - Sem webcam: o celular faz o papel da câmera, com o app Arduino IoT Remote



Monte a placa com o hub

- A** Pare os apps. Fonte USB-C na entrada PD do hub, webcam em uma porta USB-A do hub, hub na placa. Espere 30 segundos.

VOCÊ DEVE VER

No App Lab, o nome da placa de volta no canto inferior esquerdo, agora com o ícone de Wi-Fi.

SE DER ERRADO

A placa apagou? O hub não entrega energia: precisa ter Power Delivery e fonte de 3 A.



A webcam apareceu?

B

No terminal SSH da placa, liste as câmeras pelo nome.

VOCÊ DEVE VER

O nome da sua webcam na lista, com os nós **/dev/video** dela.

SE DER ERRADO

ls /dev/video* engana: mostra dois dispositivos mesmo sem câmera. São do decodificador Venus.

terminal SSH da placa

```
v4l2-ctl --list-devices
```

```
Qualcomm Venus video decoder (platform:qcom-venus):  
    /dev/video2  
    /dev/video3
```

```
See3CAM_CU30 (usb-xhci-hcd.2.auto-1):  
    /dev/video0  
    /dev/video1
```



Capturar um quadro

python/main.py do app camera-test

```
from arduino.app_utils import App
from arduino.app_peripherals.camera import Camera
from arduino.app_utils.image import compress_to_jpeg

# A primeira camera encontrada (USB antes de CSI) e usada. Sem camera, levanta excecao.
camera = Camera(resolution=(640, 480), fps=30)
camera.start()

frame = camera.capture()          # numpy array (altura, largura, canais), ordem BGR
print("Formato:", frame.shape)    # (480, 640, 3)
print("Tipo:", frame.dtype)       # uint8
print(f"Tamanho em memoria: {frame.nbytes} bytes ({frame.nbytes/1024:.0f} KB)")

jpeg = compress_to_jpeg(frame=frame, quality=90)
with open("/app/captured_image.jpg", "wb") as f:    # /app e a pasta do app na placa
    f.write(jpeg.tobytes())
print(f"JPEG salvo: {jpeg.nbytes} bytes")

camera.stop()
App.run()    # mantem o app aberto ate voce clicar em Stop
```

O código de câmera roda como app, não no terminal: é o container do app que tem o OpenCV.



Uma imagem custa caro

900 KB

um quadro 640×480
em cores

26 MB/s

passando pela memória
a 30 quadros por segundo

67%

de economia ao converter
para tons de cinza

Medido na placa. Reduzir a resolução pela metade divide os dados por quatro.



Sem webcam: o celular como câmera



Rode o exemplo **Detect Objects on Smartphone Camera** e abra a página :7000: ela mostra um código QR. Com o kit: **WS Lab 9 Celular como câmera**; o gabarito com contador e LED é **WS Lab 9 Celular e LED**.

VOCÊ DEVE VER

No celular: app **Arduino IoT Remote, Devices, +, Stream phone camera to UNO Q**, aponte para o QR. A página troca o QR pelo vídeo.

SE DER ERRADO

Celular e placa precisam estar na mesma rede Wi-Fi. O QR é lido de dentro do app, não pela câmera do celular. Se o app pedir uma senha, é a de seis dígitos que aparece embaixo do QR.



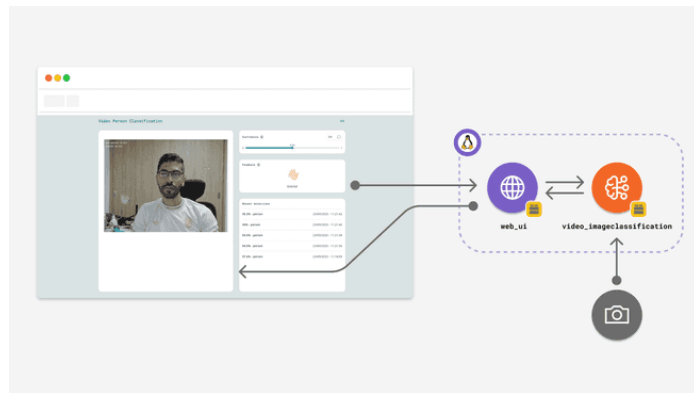


Laboratório 8

0 custo do modelo

Neste laboratório

- **Parte A:** rodar a detecção ao vivo e ler o tempo de cada inferência
- **Parte B:** trocar o modelo e medir de novo
- **Parte C:** medir com uma sonda, em vez de ler na tela
- **Parte D:** comparar quatro modelos no mesmo hardware
 - Teto teórico: $\text{FPS máximo} = 1000 \div \text{tempo de inferência em milissegundos}$
 - Este laboratório usa a webcam. Com o celular como câmera, acompanhe a demonstração do instrutor.



Detecção ao vivo

- A** Em **Inspirations**, copie **Detect Objects on Camera** com o nome **model-cost** e clique em **Run**. Com o kit: **WS Lab 9 Detecção ao vivo**.

VOCÊ DEVE VER

Em <http://<nome-da-placa>.local:7000>, o vídeo com caixas sobre os objetos.

SE DER ERRADO

No Camera Device Found? A placa não está vendo a webcam. Confira o hub.



Quanto tempo o modelo leva?

A Com o app rodando, abra **`http://<nome-da-placa>.local:4912`**. É a página do runner de modelos.

VOCÊ DEVE VER

O vídeo e a linha **Time per inference**: cerca de **370 ms** para o modelo YOLO-X nano.

SE DER ERRADO

O número não aparece? Não há nada detectável na cena. Fique na frente da câmera.



Troque o modelo

B

Pare o app. Clique no Brick, aba **AI models, Brick Configuration**, e escolha outro modelo. A escolha fica na chave **model1**: do **app.yaml**.

VOCÊ DEVE VER

Rostos: cerca de **100 ms**. Classificador de pessoa: cerca de **27 ms**.

SE DER ERRADO

Trocou e nada mudou? O modelo só é carregado na partida: Stop e Run. Com o kit: **WS Lab 8 Custo do modelo** é o app de classificação com o modelo padrão; **WS Lab 9 Classificador de pessoa** já usa o modelo de pessoa.

app.yaml (dois exemplos)

```
# deteccao de rostos, no Brick de deteccao:
bricks:
  - arduino:video_object_detection:
      model: face-detection
  - arduino:web_ui

# classificador de pessoa, no Brick de classificacao:
name: model-cost-cls
icon: 🧑
description: "Custo do modelo, classificação"
bricks:
  - arduino:video_image_classification:
      model: person-classification
```



Mesmo hardware, quatro modelos

Modelo	O que faz	Entrada	Inferência	Teto (1000 ÷ ms)	Medido
YOLO-X nano	detecta 80 classes, com caixas	416×416	370 ms	2,7/s	1,6/s
MobileNet-v2	classifica em 1000 classes	224×224	130 ms	7,7/s	3,1/s
Face detection	detecta rostos	320×256, cinza	101 ms	9,9/s	5,4/s
Person classification	pessoa ou não	224×224	27 ms	37/s	10,0/s



O modelo define o teto

370 ms

YOLO-X nano
detecção geral

27 ms

classificador de pessoa

14×

de diferença no mesmo
hardware

O Brick abre a câmera a 10 quadros por segundo. Por isso nenhum modelo passa de 10 resultados por segundo.



Medir de verdade: a sonda

ws_probe.py, na pasta do app: o laço que escuta o runner por 20 segundos

```
while time.time() - inicio < 20:
    try:
        msg = await asyncio.wait_for(ws.recv(), timeout=5)
    except asyncio.TimeoutError:
        print("nenhuma mensagem em 5 s (ha alguem na frente da camera?)")
        continue
    n += 1
    dado = json.loads(msg)
    # mensagens de resultado trazem o tempo de inferencia
    if "timeMs" in dado:
        tempos.append(dado["timeMs"])
```

```
# no terminal da placa, com o app rodando:
# docker exec <pasta-do-app>-main-1 python3 /app/ws_probe.py
# ws://ei-video-classification-runner:4912
```

O runner publica cada resultado, com o tempo de inferência em **timeMs**. O arquivo completo está no Lab 8, Passo 7.



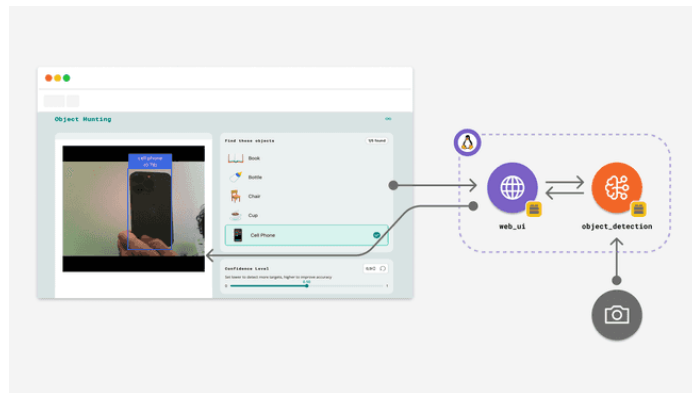


Laboratório 9

Visão em tempo real

Neste laboratório

- **Parte A:** partir do exemplo oficial de detecção ao vivo
- **Parte B:** medir a taxa de resultados de ponta a ponta
- **Parte C:** fechar o ciclo: o LED acende com uma pessoa na cena
- **Parte D:** trocar para o modelo leve e comparar
 - **Versão A:** webcam USB. **Versão B:** celular como câmera. O código de IA é o mesmo.



O callback recebe o que o modelo viu

python/main.py

```
ui = WebUI()
detection_stream = VideoObjectDetection(confidence=0.5, debounce_sec=0.0)

# O controle deslizante da pagina envia "override_th"; repassamos ao Brick
ui.on_message("override_th", lambda sid, threshold: detection_stream.override_threshold(threshold))

def send_detections_to_ui(detections: dict):
    # detections: {"person": [{"confidence": 0.94, "bounding_box_xyxy": (x1, y1, x2, y2)}], ...}
    for label, instances in detections.items():
        for det in instances:
            ui.send_message("detection", message={
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })

detection_stream.on_detect_all(send_detections_to_ui)

App.run()
```

Formato: {"person": [{"confidence": 0.94, "bounding_box_xyxy": (...)}]}



Conte os resultados por segundo

python/main.py: import time, duas variáveis e o bloco do contador no callback

```
contagem = 0
inicio = time.time()

def send_detections_to_ui(detections: dict):
    global contagem
    contagem += 1

    for label, instances in detections.items():
        for det in instances:
            ui.send_message("detection", message={
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })

    if contagem % 30 == 0:
        decorrido = time.time() - inicio
        print(f"Desempenho: {contagem / decorrido:.1f} resultados/s "
              f"({1000 * decorrido / contagem:.0f} ms por resultado) | total {contagem}")
```

No console: **Desempenho: 1.5 resultados/s**, com alguém na frente da câmera. O callback só é chamado quando há detecção.



O app de visão ganha um sketch

terminal da placa, depois sketch/sketch.ino

```
cp -r
  /var/lib/arduino-app-cli/examples/core-and-foundational/01-led-blink/03-blinking-an-led-from-python/sketch
  ~/ArduinoApps/vision-app/
ls ~/ArduinoApps/vision-app/sketch

// depois, em sketch/sketch.ino:
#include <Arduino_RouterBridge.h>

void set_led_state(bool on) {
  digitalWrite(LED_BUILTIN, on ? LOW : HIGH);  // LED_BUILTIN acende em LOW
}

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(LED_BUILTIN, HIGH);              // começa apagado
  Bridge.begin();
  Bridge.provide("set_led_state", set_led_state);
}

void loop() {}
```

Os exemplos de visão não têm a pasta **sketch/**: copie a do Blink e troque o **sketch.ino** pelo do LED.



O LED acende com uma pessoa na cena



Duas mudanças: o import passa a ser **from arduino.app_utils import App, Bridge**, e o callback ganha quatro linhas, logo depois de **contagem += 1**. Com o kit: **WS Lab 9 Detecção e LED**.

VOCÊ DEVE VER

O LED vermelho acende quando você entra na cena.

SE DER ERRADO

O LED não apaga com a cena vazia? Sem detecção o callback não é chamado. O agente do Laboratório 14 resolve.

python/main.py

```
def send_detections_to_ui(detections: dict):
    global contagem
    contagem += 1

    pessoa = "person" in detections
    # AGIR: LED acende com pessoa na cena
    Bridge.call("set_led_state", pessoa)
    if pessoa:
        print("PESSOA DETECTADA - LED ligado")
```



Dois modelos, duas respostas

	YOLO-X nano	Classificador de pessoa
O que entrega	caixas, 80 classes, vários objetos	pessoa ou não pessoa, sem caixa
Tempo por inferência	370 ms	27 ms
Resultados por segundo	cerca de 1,5	cerca de 10
Formato do callback	<code>{"person": [{...}]}</code>	<code>{"person": 0.90}</code> ou <code>{"non person": 0.97}</code>
LED ao sair da cena	só apaga no próximo resultado	apaga logo





Laboratório 10

Tamanho e restrições do modelo

Neste laboratório

- **Parte A:** calcular a memória de um modelo: parâmetros $\times$ bytes por parâmetro
- **Parte B:** o efeito da resolução de entrada
- **Parte C:** os modelos que já estão na sua placa, e quanto pesam
 - Não usa câmera nem conta: só o terminal



Pesos em números

13,6 MB

MobileNetV2 em FP32
3,4 milhões de parâmetros

3,4 MB

o mesmo modelo em INT8

13 MB

o runtime dentro
de cada arquivo **.eim**

Compare modelos pelo tempo de inferência, não pelo tamanho do arquivo.



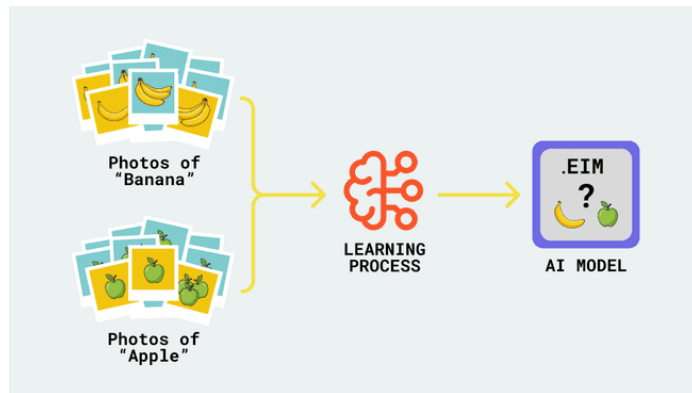


Laboratório 11

Treine o seu modelo

Neste laboratório

- Criar um projeto de **detecção de objetos** no Edge Impulse
- Coletar de 30 a 40 imagens por classe e rotular cada uma com a **caixa**
- Impulse: imagem 96×96, bloco **Object Detection**, arquitetura **FOMO MobileNetV2 0.35**
- Treinar, exportar o **.eim** e importar no Brick de detecção do App Lab
 - Imagem sem caixa não entra no treino. O Studio esconde os blocos que não combinam com os seus dados.



O mesmo tipo de app, agora com o seu modelo

python/main.py

```
def on_detections(detections: dict):
    # Formato: {"caneca": [{"confidence": 0.91, "bounding_box_xyxy": (x1, y1, x2, y2)}], ...}
    for label, instances in detections.items():
        for det in instances:
            print(f"{label}: {round(det['confidence'] * 100)} % caixa={det['bounding_box_xyxy']}")
            ui.send_message("detection", message={
                "content": label,
                "confidence": det.get("confidence"),
                "timestamp": datetime.now(UTC).isoformat(),
            })

detection_stream.on_detect_all(on_detections)

App.run()
```

Só muda o modelo que o Brick carrega.





Laboratório 12

Quantização: encolhendo o modelo

Neste laboratório

- Exportar o mesmo modelo em **float32** e em **INT8**
- Comparar precisão, tamanho e latência estimada no Studio
- Medir as duas versões na placa, na página :4912 e com a sonda
- Decidir qual implantar, com os números na mão
 - INT8 ocupa um quarto do espaço. A perda de precisão se mede, não se supõe.





Laboratório 13

Implantando e medindo o sistema

Neste laboratório

- **Parte A:** o app completo: câmera, modelo, página web e LED
- **Parte B:** CPU, memória e temperatura com o app rodando
- **Parte C:** teste sustentado de 5 minutos
- **Parte D:** monitorar de dentro do app, com `psutil` e `requirements.txt`
- **Parte E:** o relatório de desempenho



O LED só é chamado quando o estado muda

python/main.py

```
def on_detections(detections: dict):
    global frame_count, led_on
    frame_count += 1
    timestamp = time.time() - start_time

    # Registra cada detecção e envia para a página web
    for class_name, instances in detections.items():
        for det in instances:
            detections_log.append({"time": timestamp, "class": class_name,
                                   "confidence": det.get("confidence", 0)})
            ui.send_message("detection", message={"content": class_name,
                                                  "confidence": det.get("confidence"),
                                                  "timestamp": datetime.now(UTC).isoformat()})

    # Aciona o LED do MCU quando há pessoa na cena
    person = "person" in detections
    if person != led_on:
        led_on = person
        Bridge.call("set_led_state", led_on)
```

Evita uma chamada de 7 ms a cada resultado e mantém o callback leve.



Saúde do sistema pelo terminal

terminal SSH da placa

```
# Uso de CPU por processo. O runner do modelo aparece com o nome do arquivo .eim
top -bn2 -d1 | grep -E "Cpu|python|eim|gst" | tail -8

# Memoria
free -h

# Quais zonas termicas existem e qual e qual
for z in /sys/class/thermal/thermal_zone*; do echo "$(basename $z) $(cat $z/type) $(cat $z/temp)"; done

# Temperatura da CPU, em graus, com uma casa decimal
awk '{printf "%.1f n", $1/1000}' /sys/class/thermal/thermal_zone3/temp
```

No UNO Q a CPU é a zona térmica 3. O runner de modelos ocupa um núcleo inteiro.

Uma biblioteca que não vem no container

D

Bibliotecas Python extras entram pelo arquivo **python/requirements.txt**, uma por linha. O App Lab instala na partida do app.

VOCÊ DEVE VER

No console: **CPU 1% | RAM 19%**, uma linha a cada 5 segundos.

SE DER ERRADO

No module named 'psutil'? Faltou o **requirements.txt**, ou a placa estava sem internet na partida.

terminal da placa

```
echo "psutil" >
~/ArduinoApps/monitor-app/python/requirements.txt

# python/main.py
from arduino.app_utils import App
import psutil, time

def monitor_loop():
    cpu = psutil.cpu_percent(interval=1)
    mem = psutil.virtual_memory().percent
    print(f"CPU {cpu:.0f}% | RAM {mem:.0f}%")
    time.sleep(4)

App.run(user_loop=monitor_loop)
```





Laboratório 14

Agente de borda

Neste laboratório

- Inferência responde: **o que estou vendo agora?**
- Um agente responde: **o que venho vendo nos últimos segundos? Devo agir?**
- O modelo é o mesmo. Muda a camada de decisão em volta dele.
 - Uma pessoa que só passa não deve disparar o alarme
 - Este laboratório usa a webcam: o modelo de pessoa não tem versão para o celular neste material



Três estados

Estado	Sai quando	Vai para
IDLE	peessoa presente há 3 s e vista na maior parte das observações desse período	ALERT
ALERT	ninguém visto há 2 s	COOLDOWN
COOLDOWN	passaram 10 s desde o fim do alerta	IDLE



A memória do agente

python/main.py

```
class DetectionTracker:
    """Memoria e regras de decisao do agente."""

    CONFIDENCE_THRESHOLD = 0.6    # confianca minima para contar como pessoa
    SUSTAIN_SECONDS = 3.0         # presenca continua necessaria para alertar
    CLEAR_SECONDS = 2.0           # ausencia necessaria para encerrar o alerta
    COOLDOWN_SECONDS = 10.0       # espera minima entre dois alertas
    HISTORY_SECONDS = 10.0        # quanto tempo de historico e mantido

    def __init__(self):
        self.history = []          # lista de (timestamp, pessoa_presente: bool)
        self.state = "IDLE"        # IDLE, ALERT ou COOLDOWN
        self.presence_start = None  # quando a presenca atual comecou
        self.last_seen = 0.0        # ultima vez que uma pessoa foi vista
        self.alert_start = None
        self.cooldown_start = 0.0
        self.total_alerts = 0
```

As cinco constantes do topo são a personalidade do agente.



Teste o agente

A Crie o app **smart-agent** com o Brick **Video Image Classification** e o modelo de pessoa. Com o kit: **WS Lab 14 Agente de borda**. Clique em **Run**.

VOCÊ DEVE VER

- 1) Parado na frente: alerta em cerca de 3 s.
- 2) Saiu: encerra em 2 s.
- 3) Passou rápido: nada.
- 4) Voltou logo depois: nada por 10 s.

SE DER ERRADO

Para ver o enquadramento, abra a página :**4912**. Este app não tem página própria.





Laboratório 15

Benchmarking e confiabilidade

Neste laboratório

- **Parte A:** um benchmark com aquecimento, repetição e percentis
- **Parte B:** registrar CPU, memória e temperatura ao longo do tempo
- **Parte C:** provocar falhas: câmera desconectada, rede fora do ar
- **Parte D:** o relatório de referência
 - Um watchdog leva o sistema para o estado seguro quando o modelo para de responder



Saúde do sistema ao longo do tempo

~/monitor.sh na placa

```
#!/bin/bash
# Registra CPU (usuario+sistema), memoria usada e temperatura da CPU a cada 6 ou 7 s
echo "Hora,CPU%,Mem_MB,Temp_C"
while true; do
    CPU=$(top -bn2 -d1 | grep 'Cpu(s)' | tail -1 | awk '{print $2 + $4}')
    MEM=$(free -m | awk '/Mem:/ {print $3}')
    TEMP=$(cat /sys/class/thermal/thermal_zone3/temp 2>/dev/null || echo 0)
    TEMP_C=$(echo "scale=1; $TEMP / 1000" | bc)
    echo "$(date +%H:%M:%S), $CPU, $MEM, $TEMP_C"
    sleep 5
done

# para rodar e gravar:
# bash ~/monitor.sh | tee ~/system_log.csv
```

Roda até você apertar Ctrl+C. O arquivo **system_log.csv** vai para o relatório.

O watchdog

python/main.py do app smart-agent

```
def watchdog_check():
    """Se o modelo parou de responder, leva o sistema para o estado seguro."""
    global safe_mode, led_on
    parado = time.time() - last_result_time > WATCHDOG_TIMEOUT
    if parado and not safe_mode:
        safe_mode = True
        print(f"WATCHDOG {time.strftime('%H:%M:%S')}: sem resultados ha "
              f"{WATCHDOG_TIMEOUT:.0f} s. Entrando em modo seguro.")
        tracker.state = "IDLE"          # o agente esquece o alerta em curso
        tracker.presence_start = None    # e volta a exigir presença sustentada
        led_on = False
        try:
            Bridge.call("set_led_state", False)
        except Exception as e:
            print(f"WATCHDOG: falha ao falar com o MCU: {e}")
    elif not parado and safe_mode:
        safe_mode = False
        print(f"WATCHDOG {time.strftime('%H:%M:%S')}: resultados voltaram. Operacao normal.")
```

Apaga o LED, zera **led_on** e devolve o agente a IDLE. O software e o mundo físico mudam juntos.





Fechamento

O que você leva daqui

O que você fez

- Programou os **dois processadores** de uma placa e fez um comandar o outro
- Rodou modelos de IA **na placa**, sem nuvem
- **Mediu**: 7 ms para falar com o microcontrolador; de 27 ms a mais de 1 segundo para o modelo pensar
- Fechou o ciclo **sentir, pensar, agir**
 - Os laboratórios que ficaram de fora estão no manual e nestes slides: com uma placa, dá para fazer todos



Para continuar

- **Manual do Laboratório de IA Edge 1:** os 16 laboratórios completos
- **Exemplos do App Lab:** dezenas de apps prontos para copiar e editar
- **Edge Impulse:** treine um modelo com os seus dados
- **docs.arduino.cc**, seção UNO Q
- **Repositório do workshop:** github.com/joaodullius/workshop-edge-ai-uno-q, com o manual, estes slides e o kit de apps





Obrigado!